posco

설비 고장을 예측하는 AI 데이터 분석

예외처리 및 종합 실습

견고한 예외처리

PART 2

견고한 예외처리

else와 finally부터 with open,
bare except 회피, raise 검증까지
실무에서 통하는 예외처리 패턴을 정리합니다.

01 **else와 finally 구성**
else와 finally / 리소스 정리

02 **안전한 파일 처리 구성**
with open 활용 / CSV 예외처리

좋은 예외처리 원칙 구성
bare except / raise 활용

이 단원의 실데이터 — ICT 검사

예외처리로 다룰 현장 검사 데이터를 먼저 이해

공정 — ICT 검사

ICT는 제조 부품의 전기 특성을 측정해 규격 안에 드는지 판정하는 검사 공정이다.

핵심 값과 결측

부품명별로 측정값·기준값·상한/하한을 기록한다. 실제 현장 데이터에는 빈칸(결측)·형식 오류가 섞여 있다.

왜 예외처리

측정값이 비거나 문자·범위 밖이면 형변환·인덱스 오류가 난다. try·except로 불량 줄을 건너뛰고 견고하게 처리한다.

출처: KAMP 제조AI 데이터셋 — ICT 검사 (중소기업기술정보진흥원·KAIST). 교육용 발췌 + 품질문제 주입.

이 파일에 섞여 있는 불량 4종

> 세 가지는 예외로 잡히고, 마지막 하나는 예외 자체가 발생하지 않음

검사ID	문제	예외로 잡히는가
9001	측정값이 문자 “결측”	**ValueError — 잡힘**
9002	측정값이 빈칸	**ValueError — 잡힘**
9004	칸이 3개뿐인 짧은 줄	**IndexError — 잡힘**
9003	측정값 -999	**예외 미발생 — raise 필요**

posco

else와 finally

> **else**는 성공했을 때만 실행되는 블록
>
> **finally**는 무조건 마지막에 실행되는 블록

지난 시간에 배운 **try / except**에 두 형제가 더 붙습니다.

else는 조건문의 else와 같은 단어로, try에서 예외가 하나도 발생하지 않았을 때만 실행됩니다.

finally는 '마지막으로'라는 뜻 그대로, 성공하든 실패하든 무조건 마지막에 실행됩니다.

이 성질 덕분에 finally는 **파일 닫기처럼 반드시 처리해야 할 마무리 작업**에 자주 쓰입니다.

else와 finally 코드

CODE

네 블록 전체 동작

```
try:
    temp = float(text)
except ValueError:
    print("변환 실패")
else:
    print("변환 성공:", temp)   # 예외가 없을 때만
finally:
    print("종료")             # 성공·실패 무조건
```

'62.5' 입력 → 변환 성공: 62.5 / 종료

'결측' 입력 → 변환 실패 / 종료 — finally는 어느 경로에서도 실행

posco

네 블록의 실행 정리

try·except·else·finally — 네 블록이 어떤 경로로 실행되는지 정리합니다.

CASE 01

성공 경로

try를 끝까지 실행한 뒤
else로 이동하고,
마지막에 **finally**가 실행됩니다.

CASE 02

실패 경로

try가 중단되면
except로 이동하고,
마지막에 **finally**가 실행됩니다.

CASE 03

자주 쓰는 것

넷을 다 쓸 필요는 없습니다.
초보 단계에서는
finally가 가장 유용합니다.

posco

리소스 정리가 필요한 이유

> 열어둔 파일을 닫지 않으면 **데이터 손실과 자원 낭비**
>
> 정리는 **선택이 아니라 의무**

파일을 열면 운영체제가 우리 프로그램에 **연결 통로**를 내어줍니다.

이 통로는 **한정된 자원**이라서, 닫지 않고 계속 쌓이면 결국 고갈됩니다.

게다가 파일에 쓴 내용은 닫는 순간 실제로 저장 — 닫지 않으면 저장 대기 중인 데이터 소실

읽는 도중 예외가 나면 그 아래 close는 실행되지 않음 — 그래서 닫기를 finally에 배치

PRACTICE

실습 1. finally로 파일 안전하게 닫기

> 목표
>
> **try-finally로 오류가 나도 파일을 반드시 닫기**

단계

① try 블록에서 파일을 열어 처리
② 처리 도중 오류가 날 수 있음을 가정
③ finally 블록에 close를 넣어 오류 여부와 상관없이 닫기
④ 일부러 오류를 내도 finally가 실행되는지 확인

예상 결과 오류가 나도 파일이 안전하게 닫힘

PRACTICE · ANSWER

실습 1. finally로 파일 안전하게 닫기 정답

CODE · 정답

```
f = open("data/09_ict_inspection.csv", "r", encoding="utf-8")
try:
    first = f.readline()
    print(first.strip())   # 헤더 줄
finally:
    f.close()   # try에서 오류가 나도 반드시 닫힘
print("파일이 안전하게 닫힘")
```

posco

with open으로 더 안전하게

> with open을 쓰면 자동으로 닫힘 — 파일 처리의 표준

CODE

블록이 끝나면 자동으로 닫힘

```
with open(name) as f:
    data = f.readlines()   # 블록 끝나면 자동으로 닫힘
```

with open as f 형태로 쓰면 들여쓴 블록이 끝나는 순간 파이썬이 파일을 자동으로 닫음

블록 밖에서 쓸 값은 블록 안에서 리스트나 변수에 담아두기 — 밖에서는 이미 닫힌 상태

자동인 것은 닫기뿐 — 파일 없음은 여전히 try-except로 별도 처리

앞으로 파일을 열 때는 with open을 기본으로 사용

PRACTICE

실습 2. CSV를 안전하게 읽기

목표

바깥과 안쪽 두 곳에 예외처리를 두어 csv 합계 구하기

단계

① 파일 열기를 try로 감싸 파일 없음에 대비
② 없으면 빈 결과를 돌려 흐름 유지
③ 한 줄씩 돌며 측정값 칸의 형변환을 다시 try로 감싸기
④ 숫자로 못 바꾸는 칸은 건너뛰고 합계만 누적

예상 결과 파일이 없어도 멈추지 않고, 숫자인 칸만 합계에 반영

PRACTICE · ANSWER

실습 2. CSV를 안전하게 읽기 정답

CODE 이중 예외처리

```
try:
    with open("data/09_ict_inspection_dirty.csv", encoding="utf-8") as f:
        rows = f.readlines()[1:]
except FileNotFoundError:
    rows = []
total = 0.0
for line in rows:
    try:
        total += float(line.split(",")[2])   # 측정값(2열)
    except ValueError:
        pass                                  # 숫자로 못 바꾸는 칸은 건너뜀
print(round(total, 2))   # 27594.74
```

바깥 try-except는 파일이 없는 경우를 차단

안쪽 try-except는 각 칸을 숫자로 못 바꾸는 경우를 차단

두 위험을 각각 다른 위치에서 처리하는 것이 핵심 — 21행 중 17행이 합계에 반영, 4행은 건너뜀

posco

반복문 안에서의 예외처리

CODE — 불량 줄은 continue로 건너뛰기

```
try:
    temp = float(row_temp)
except ValueError:
    continue   # 불량 줄은 건너뛰고 다음 줄로
```

각 줄의 변환을 try로 감싸 한 줄의 예외가 그 줄에만 영향

except에서 불량을 센 뒤 continue로 다음 줄로 이동 — 반복은 중단 없이 계속

try를 반복문 안쪽에 두는 것이 핵심 — 바깥에 두면 첫 불량에서 전체 중단

건너뛸 때의 세 가지 선택

> 같은 "건너뛰기"라도 남기는 정보가 다름

선택 01

pass

아무것도 하지 않고

그냥 통과
무엇이 빠졌는지 알 수 없음

선택 02

continue

이번 줄을 멈추고
다음 줄로 이동
반복문 안에서만 사용

권장

기록 후 continue

어떤 데이터가 왜 빠졌는지 추적 가능
줄 번호와 이유를 모은 뒤
다음 줄로 이동

같은 방식으로 처리할 예외 묶기

CODE — 괄호로 묶어 한 번에 처리

```
try:
    v = float(cols[2])        # 측정값
except (ValueError, IndexError):
    skipped += 1              # 두 예외 모두 같은 처리
    continue
```

대처 방법이 같은 예외는 괄호로 묶어 하나의 except로 처리

글자가 섞인 줄과 칸이 부족한 줄 모두 "건너뛰기"로 같은 대응

나누는 기준은 예외 종류가 아니라 대처가 다른가 — 대처가 같으면 묶기

PRACTICE

실습 3. 반복문에서 불량 줄 건너뛰기

목표

반복문 안에서 불량 줄만 건너뛰고 정상값만 합산하기

단계

① 여러 측정값(일부는 숫자가 아님)을 반복
② try에서 float로 변환
③ 변환 실패(ValueError) 시 continue로 그 줄만 건너뛰기
④ 정상 값만 합계에 더해 출력

예상 결과 불량 줄은 건너뛰고 정상값만 합산

PRACTICE · ANSWER

실습 3. 반복문에서 불량 줄 건너뛰기 정답

CODE · 정답

```
with open("data/09_ict_inspection_dirty.csv", "r", encoding="utf-8") as f:
    rows = f.readlines()
total = 0.0
skipped = 0
for line in rows[1:]:                  # 헤더 제외
    cols = line.strip().split(",")
    try:
        total += float(cols[2])        # 측정값(2열)
    except (ValueError, IndexError):
        skipped += 1
        continue                       # 불량 줄은 건너뜀
print("합계:", round(total, 1), "건너뜀:", skipped)
```

뒤에서 배울 **raise 검증**이 필요한 이유입니다.

PRACTICE

실습 4. 여러 파일 묶어 처리하기

> 목표
>
> **여러 파일을 묶어 처리하되 없는 파일은 건너뛰기**

단계

① 여러 파일 이름을 반복
② try에서 파일을 열어 처리
③ 없는 파일(FileNotFoundError) 시 continue로 건너뛰기
④ 처리한 파일 수를 세어 출력

예상 결과 없는 파일은 건너뛰고 있는 파일만 처리

PRACTICE · ANSWER

실습 4. 여러 파일 묶어 처리하기 정답

CODE · 정답

```
names = ["data/09_ict_inspection.csv",
         "data/09_ict_inspection_dirty.csv",
         "data/09_없는파일.csv"]
done = 0
for name in names:
    try:
        with open(name, encoding="utf-8") as f:
            f.readlines()
        done += 1
    except FileNotFoundError:
        continue   # 없는 파일은 건너뜀
print("처리한 파일:", done)   # 2
```

except와 continue가 없다면 첫 없는 파일에서 멈춘다는 점을 직접 확인해 보세요.

bare except의 위험

> 모든 예외를 뭉뚱그려 잡는 except는 **진짜 버그를 숨김**
>
> 잡을 예외는 **구체적으로 지정**

except 뒤에 아무것도 안 붙이고 그냥 except만 써도 동작합니다.
모든 예외를 다 잡는 이것을 **bare except**라고 부르며, 피해야 할 나쁜 습관입니다.

예상한 예외뿐 아니라 변수 오타 같은 진짜 버그까지 잡아 조용히 통과 — 경고등을 테이프로 가리는 격

사용자의 강제 종료 신호까지 잡아 프로그램이 멈추지 않을 위험

except 뒤에는 예상되는 예외만 구체적으로 지정하고, 새 예외를 만날 때마다 하나씩 추가

나쁜 예외처리 vs 좋은 예외처리

> 세 가지 항목으로 나쁜 예와 좋은 예를 나란히 비교합니다.

구분	나쁜 예	좋은 예
예외 지정	전부 잡기	**구체적 지정**
try 범위	전체 감쌈	**위험한 줄만**
문제 처리	무시	**줄 번호 기록**

posco

좋은 예외처리 4원칙

> 예상한 문제는 우아하게 처리 · 예상 못한 문제는 정직하게 드러내기 — 네 단어로 정리

01 구체적으로

bare except 대신 **예상되는 예외만**
정확히 지정합니다.

02 작게

위험한 코드만 try에 담아
오류 위치를 명확히 합니다.

03 기록

어떤 데이터가 왜 빠졌는지 줄 번호와 이유를 기록
예외가 나지 않는 문제도 함께 기록해야 분석 신뢰도 확보

04 드러내기

예상 못한 예외는 잡지 말고 드러내서
코드를 고치게 둡니다.

posco

에러 정보 꺼내 쓰기 — as e

CODE — as e로 원인 메시지 활용

```
try:
    temp = float(v)
except ValueError as e:
    print(e)   # e에 예외 원인 메시지
```

except 뒤에 **as e**를 붙이면 **e에 예외 정보**가 담깁니다.

e를 출력하면 트레이스백 맨 아랫줄에서 봤던 그 **구체적인 설명**이 나옵니다.

단순히 '문제 발생'이 아니라 **어떤 값이 왜 문제인지**를 기록할 수 있습니다.

posco

as e 활용 포인트

> e가 주는 원인 정보를 어떻게 활용할지 세 가지 관점으로 정리합니다.

POINT 01

맥락 결합

내가 아는 **줄 번호**와
파이썬이 주는 **원인**을 합치면
더 정확한 기록이 됩니다.

POINT 02

사용자용

사용자에겐
'온도 값이 올바르지 않습니다'
처럼 이해하기 쉬운
우리말 안내를 제공합니다.

POINT 03

개발자용

기록에는
e의 구체적 원인을 남겨
나중에 원인을 추적합니다.

raise로 직접 예외 발생시키기

CODE 규칙 어긴 값에 직접 예외 발생

```
if not (0.0 <= v <= 5000.0):
    raise ValueError("정상 범위 밖")   # 우리 규칙을 벗어나면 직접 예외 발생
```

측정값 –999는 파이썬 입장에선 정상 숫자 — 아무 예외도 발생하지 않음

하지만 ICT 검사에서 –999는 설비가 값을 못 읽었다는 오류 표시

우리가 정한 범위를 벗어나면 raise로 직접 ValueError 발생 — 이렇게 일으킨 예외도 try-except로 포착

raise는 파이썬이 못 잡는 우리만의 규칙을 코드로 강제하는 도구

raise 활용 포인트

> 실무 검증 함수에 raise를 넣을 때 짚어야 할 세 가지

포인트	내용	ICT 검사 예
현실 규칙	항목마다 현실적인 정상 범위 존재	**측정값 0 ~ 5000**
역할 분리	검증 함수는 예외만 발생, 대응은 호출부	**check_value() ↔ try-except**
예외 종류	값이 부적절한 경우이므로 ValueError	**raise ValueError(...)**

PRACTICE

실습 5. 함수 안에서 입력값 검증하기

목표

정상 범위를 벗어난 값을 함수 안에서 직접 차단하기

단계

① 정상 범위를 인자로 받는 검증 함수를 정의
② 함수 안에서 형변환으로 값 확인
③ 정해둔 범위를 벗어나면 raise로 직접 예외 발생
④ 부르는 쪽에서 try-except로 잡아 안내

예상 결과 정상값은 통과 / 문자·범위 밖 값은 각각 걸러짐

PRACTICE · ANSWER

실습 5. 함수 안에서 입력값 검증하기 정답

CODE · 정답

```
def check_value(text, lo=0.0, hi=5000.0):
    v = float(text)                        # 실패 시 ValueError
    if not (lo <= v <= hi):
        raise ValueError(f"정상 범위 밖: {v}")
    return v

for t in ["128.5", "결측", "-999"]:
    try:
        print("정상값:", check_value(t))
    except ValueError as e:
        print("걸러짐:", e)
```

128.5는 통과, "결측"은 변환 실패, –999는 범위 밖으로 차단

검증 함수는 raise로 예외만 발생시키고, 부르는 쪽이 try-except로 대응을 결정

raise 검증 전과 후 — 평균이 달라진다

> –999 한 줄이 전체 평균을 끌어내리는 것을 수치로 확인

구분	유효 개수	측정값 평균
범위 검증 전	17건	**1623.22**
범위 검증 후	16건	**1787.11**
차이	–999 한 줄	**+163.89**

posco

정상 파일과 불량 파일 — 같은 코드, 다른 결과

> 견고하게 짠 코드는 두 파일 모두 멈추지 않고 처리

파일	전체 행	유효 / 제외
09_ict_inspection.csv	17행	**유효 15 / 제외 2**
09_ict_inspection_dirty.csv	21행	**유효 16 / 제외 5**
공통	코드 동일	**어느 쪽에서도 중단 없음**